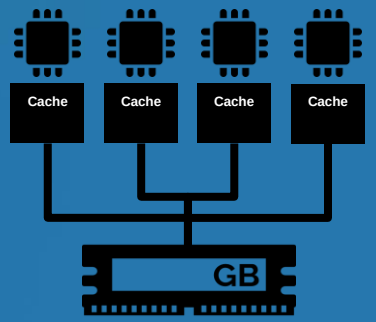
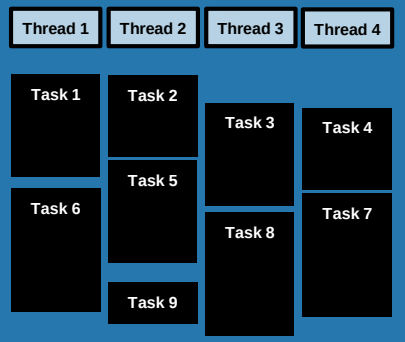
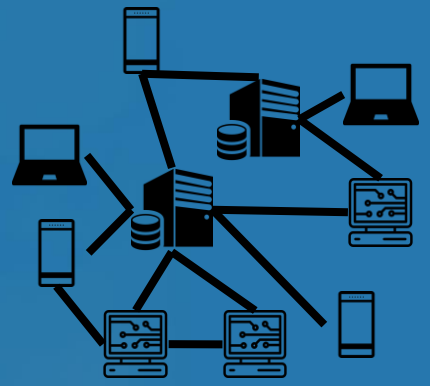
Parameterized Verification 2024

Lecture 3







The Parameterized Model-Checking Problem (PMCP)

Let

- $\mathcal{P}$ be a set of processes (e.g., all finite-state processes $P = (Q, Q_0, \Sigma, \delta, \lambda)$)
- $card$ be a synchronization constraint (e.g., $card = \{1\}$ for pairwise rendezvous)
- $\mathcal{F}$ be a set of specifications (e.g., reachability of states from $\uparrow R$ for some $\mathrm{R} \subseteq Q^n$)

Then the parameterized model-checking problem (PMCP) for $\mathcal{P}, card$ and $\mathcal{F}$ is defined by

Input: $P \in \mathcal{P}$ and $\varphi \in \mathcal{F}$

Output: "Yes" if $\forall n \in \mathbb{N}: (P^n_{card} \vDash \varphi)$, "No" otherwise.

How can such problems be solved?

Reachability Specifications in Parameterized Systems (II)

For $R \subseteq Q^n$, let $\uparrow R = \bigcup_{m \geq n} \{s \in Q^m \mid s \text{ contains } s' \text{ for some } s' \in R\}$.

We call $\uparrow R$ the (upward) closure of R.

Examples:

$\uparrow \{(q)\} = \{(q)\} \cup \{(q, q'), (q', q) \mid q' \in Q\} \cup \{(q, q', q''), (q', q, q''), (q', q'', q) \mid q', q'' \in Q\} \cup \cdots$

$\uparrow \{(q_1, q_2)\} = \{(q_1, q_2), (q_2, q_1)\} \cup \{(q_1, q_2, q), (q_1, q, q_2), \ldots \mid q \in Q\} \cup \cdots$

For parameterized correctness (in completely symmetric systems),
we are interested in reachability of $\uparrow R$, for some $R \subseteq Q^n$.

$\uparrow R$ contains states of arbitrary size,
even though $R \subseteq Q^n$ for fixed n.

Decidability Results: Broadcast Protocols

Counter System of a Broadcast Protocol (II)

Let $P = (Q, Q_0, \Sigma, \delta, \lambda)$ be a process based on actions Σ_{int} and Σ_{sync}, and let $card = \mathbb{N}_0$, i.e., we consider broadcast synchronization.

The counter system for P^n is the transition system $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ with

- $C = \{0, \dots, n\}^{|Q|}$, i.e., each abstract state c $\in C$ can also be seen as a function c: $Q \to \{0, \dots, n\}$
- $C_0 = \{c \in C \mid c(q) = 0 \text{ if } q \notin Q_0\}$
- Δ_{abs} contains a transition $abs(s) \xrightarrow{a} abs(s')$ iff $s \xrightarrow{a} s'$ is a transition in P^n

(these abstract systems are usually considered without state labels)

Expressing Broadcast Receive as a Matrix Multiplication

Operations in point 1 and 3 of the computation are simple additions of vectors.

The operation in point 2 can be represented as a matrix multiplication:

c_2 with $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$ is the same as $c_2 = M_a \cdot c_1$ for the matrix

$$M_a = \begin{pmatrix} m_{0,0} & \cdots & m_{0,|Q|-1} \\ \vdots & \ddots & \vdots \\ m_{|Q|-1,0} & \cdots & m_{|Q|-1,|Q|-1} \end{pmatrix} \text{ with } m_{k,l} = \begin{cases} 1 \text{ if } q_l \xrightarrow{a??} q_k \\ 0 \text{ otherwise} \end{cases}$$

Note: each column of M_a is a unit vector, and the ith column determines where processes from state q_i go upon a broadcast receive

Counter Systems for Model-Checking Large Systems

Counter systems (with or without matrix form) allow us to model-check large systems more efficiently:

we can modify the model-checking algorithm (on the right) to use abstract states to represent $Reached$, and the approach shown before to compute $\delta(Reached)$

However, until now it does not enable parameterized verification:

- either we start with a fixed-size system, then we only verify that size
- or we start with initial states of all sizes (which is infinite even in counter system), then we can't compute $\delta(Reached)$

To check reachability of R in M, we can for example use the following algorithm, where $\delta(Reached)$ is the set of all states that are in $Reached$, or are reachable in one step of δ :

1. Let $Reached = Q_0$
2. Let $Reached' = \delta(Reached)$
3. If $R \cap Reached' \neq \emptyset$ then return "reachable"
4. If $Reached' \neq Reached$ then {let $Reached = Reached'$; goto 2}
5. Return "unreachable"

How can we extend it to support parameterized verification?

Counter System of a Parameterized System

Let $P = (Q, Q_0, \Sigma, \delta, \lambda)$ be a process based on actions Σ_{int} and Σ_{sync}, and let $card = \mathbb{N}_0$, i.e., we consider broadcast synchronization.

The counter system for $sys(P, card)$ is the transition system $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ with

- $C = \mathbb{N}^{|Q|}$, i.e., each abstract state $c \in C$ can also be seen as a function $c: Q \to \mathbb{N}$
- $C_0 = \{c \in C \mid c(q) = 0 \text{ if } q \notin Q_0\}$
- Δ_{abs} contains a transition $abs(s) \xrightarrow{a} abs(s')$ iff $s \xrightarrow{a} s'$ is a transition in P^n for some $n \in \mathbb{N}$

Essentially: starts in initial state of arbitrary size, then takes transitions in instance of that size.

(unless mentioned otherwise, we always consider parameterized systems where number of processes does not change during a run)

How can we efficiently compute with an infinite number of states?

Properties of Well-Quasi-Orders

Let S be an infinite set, $\lesssim \subseteq S \times S$ a wqo on S.

Then:

- the upward closure of a set $S' \subseteq S$, denoted $\uparrow S'$, is the set $\{s \in \mathrm{S} \mid \exists s' \in S' : s' \lesssim s\}$
  (**Note**: this is a generalization of the upward closure $\uparrow R$ on states in parameterized systems we have seen before; we will later show when they coincide)
- we call a set $S' \subseteq S$ upward-closed if $\uparrow S' = S'$
- we call $B \subseteq S$ a basis of an upward-closed set S' if $\uparrow B = S'$
- if S' is upward-closed, then it has a finite minimal basis,
  i.e., a basis consisting only of elements $s \in S$ such that $\nexists s' \in S : s' \preccurlyeq s$.
  (if it was not finite, then the set of minimal elements would constitute an infinite antichain)

Well-Quasi-Order on States in Counter System

Theorem:
Let $S = \mathbb{N}^k$ for some $k \in \mathbb{N}$. For $s = (i_1, \dots, i_k) \in S$, let $s(j)$ denote i_j, the jth component of s. Then the relation $\preccurlyeq \subseteq S \times S$ defined by

$$s \preccurlyeq s' \iff \forall j \in \{1, \dots, k\}: s(j) \leq s'(j)$$

is a well-quasi-order on S. It is also called component-wise ordering.

Proof: Clearly, $\preccurlyeq$ is reflexive and transitive.

To see that the third property of a wqo holds, assume that we have an infinite sequence $s_0, s_1, s_2, \dots$ that does not contain a pair $s_i \preccurlyeq s_j$ with $i < j$. Such a sequence must contain either a subsequence

- with infinitely many strictly decreasing elements, i.e., $s_i \succcurlyeq s_j \wedge s_i \neq s_j$ for $i < j$, or
- with infinitely many incomparable elements, i.e., $\neg(s_i \preccurlyeq s_j) \wedge \neg(s_i \succcurlyeq s_j)$ for $i < j$

The first is not possible, since for any s_i there are only finitely many elements strictly smaller than s_i.

The second is also not possible: s_i and s_j are incomparable if there are k,l with $s_i(k) < s_j(k)$ and $s_i(l) > s_j(l)$. In particular, an element s_j in a sequence of pairwise incomparable elements must be strictly smaller in at least one component than all preceding elements. For any s_0, this is only possible finitely often.

Preview and Learning Goals for this Lecture

Topics:

- Model Checking Broadcast Protocols
- Example application of model checking algorithm
- Parameterized model checking:
  - for rendezvous systems
  - for general safety properties

Techniques:

- Well-structured Transition Systems (WSTSs)
- Expressing safety properties as finite automata
- Parameterized model checking based on finite automata

Model Checking Broadcast Protocols

Let $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ be the counter system of a parameterized system $sys(P, \mathbb{N}_0)$.

For $R' \subseteq C$, the set of immediate predecessors of R' is

$$pred(R') = \{c \mid \exists c' \in R' \; \exists a \in \Sigma : c \xrightarrow{a} c'\} = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R' : \; c \xrightarrow{a} c'\}.$$

I.e., $pred(R')$ is the set of all abstract states that can reach R' in one transition.

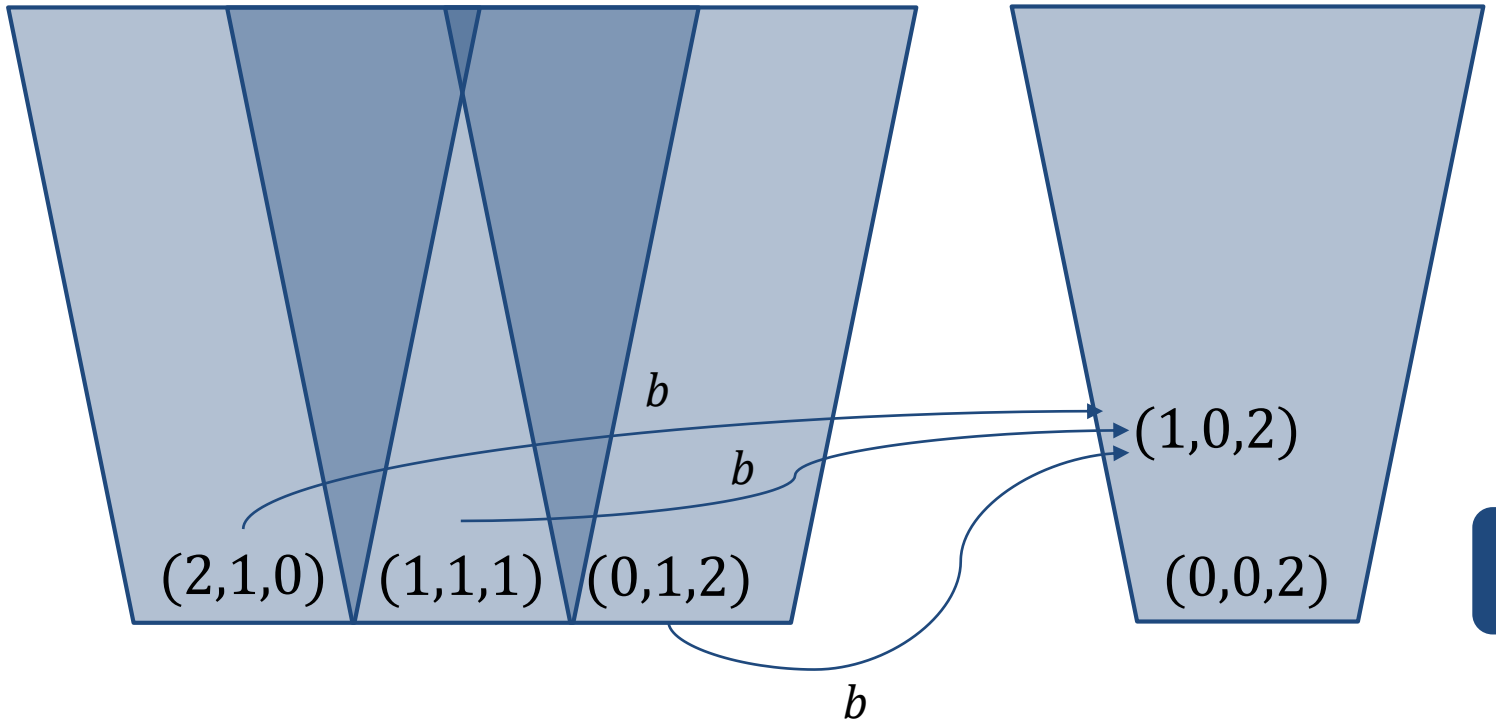
Idea: if R' is upward-closed, it has a finite basis B', and we can compute a finite basis B of $pred(R')$

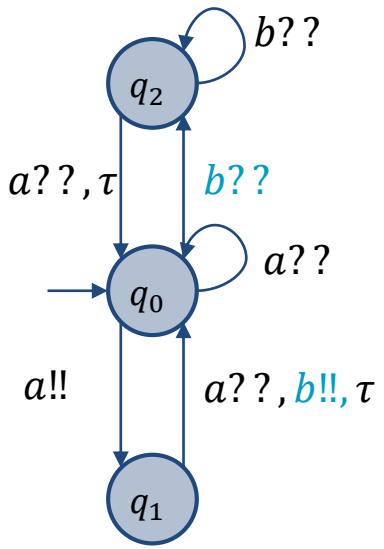
However: if $R' = \uparrow B'$, then we may have $\uparrow pred(B') \neq pred(R')$

Model Checking Broadcast Protocols (II)

We want to check if we can reach a state with 2 or more processes in q_2.

To this end, we need to compute $pred(R')$, with $R' = \uparrow \{(0,0,2)\}$





$pred(\{(0,0,2)\}) = \emptyset$,
but $pred(R') \neq \emptyset$

Model Checking Broadcast Protocols (III)

Let $B = (C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ be the counter system of a parameterized system $sys(P, \mathbb{N}_0)$.

For $R' \subseteq C$, the set of immediate predecessors of R' is

$$pred(R') = \{c \mid \exists c' \in R' \; \exists a \in \Sigma : c \xrightarrow{a} c'\} = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R' \; c \xrightarrow{a} c'\}.$$

I.e., $pred(R')$ is the set of all abstract states that can reach R' in one transition.

Then:

1. if R' is upward-closed, then $pred(R')$ is upward-closed (and both have a finite minimal basis)
2. a finite minimal basis of $pred(R')$ can be computed from a finite minimal basis of R'

Model Checking Broadcast Protocols (IV)

For $R' \subseteq C$, the set of immediate predecessors of R' is

$$pred(R') = \{c \mid \exists c' \in R'\ \exists a \in \Sigma: c \xrightarrow{a} c'\} = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R':\ c \xrightarrow{a} c'\}.$$

Proof idea:

1. Claim: if R' is upward-closed, then $pred(R')$ is upward-closed

$$pred(R') = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R':\ c \xrightarrow{a} c'\}$$
$$= \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R': c' = M_a \cdot (c - u_i) + u_j \wedge c(i) \geq 1\} \qquad \text{(where } q_i \xrightarrow{a!!} q_j\text{)}$$

Since $M_a \cdot d \preccurlyeq M_a \cdot d'$ and $c + d \preccurlyeq c + d'$ whenever $d \preccurlyeq d'$, each set of the union is upward-closed.

Finally, a finite union of upward-closed sets is again upward-closed.

2. Claim: a finite minimal basis of $pred(R')$ can be computed from a finite minimal basis of R'

One can show that the finite minimal basis of $pred(R')$ are the minimal elements of

$$\bigcup_{a \in \Sigma} \{c \mid \exists c' \in basis(R'): \left[\left(c' = M_a \cdot (c - u_i) + u_j\right) \vee \left(c'(j) = 0 \wedge c' + u_j = M_a \cdot (c - u_i) + u_j\right)\right] \wedge c(i) \geq 1\}$$

case: $c \xrightarrow{a} c'$ case: $c \xrightarrow{a} (c' + u_j)$

Model Checking Broadcast Protocols (V)

This leads us to the following algorithm for checking reachability of $R = \uparrow B$:

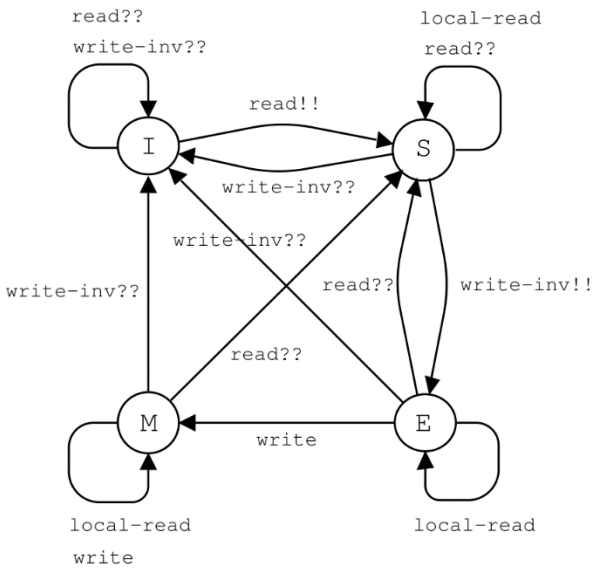
1. Let $Initial = \{c \in C \mid c(q) = 0 \text{ if } q \notin Q_0\}$
2. Let $Reached = \uparrow B$ — we start computation with unsafe states
3. Let $Reached' = pred(Reached) \cup Reached$ — we compute backwards with $pred$
4. If $Initial \cap Reached' \neq \emptyset$ then return "reachable" — check if initial states have been reached
5. If $Reached' \neq Reached$ then {let $Reached = Reached'$; goto 3} — check if any new states have been reached; if so, repeat
6. Return "unreachable" — if we reached a fixpoint, but no initial states, then $\uparrow R$ is unreachable

$Reached$ is always upward-closed and can be represented by a finite minimal basis

Example: Model Checking a Simplified MESI Protocol

Cache coherence protocol with following states:

- M for modified (cache contents has been changed)
- E for exclusive (no other process holds this cache line)
- S for shared (multiple processes hold the cache line, it is unchanged)
- I for invalid (the contents of the cache is not guaranteed to be valid because other processes may have changed it)
- Initial state: all processes are in I



Important safety properties (for state vector (M, E, S, I)):

- there should not be more than one process in M or E at the same time, i.e., $\uparrow\{(2,0,0,0), (1,1,0,0), (0,2,0,0)\}$ are unsafe states
- a cache line should not be modified by one process (M) and in shared state for another (S), i.e., $\uparrow\{(1,0,1,0)\}$ are unsafe states

Example: Model Checking a Simplified MESI Protocol (II)

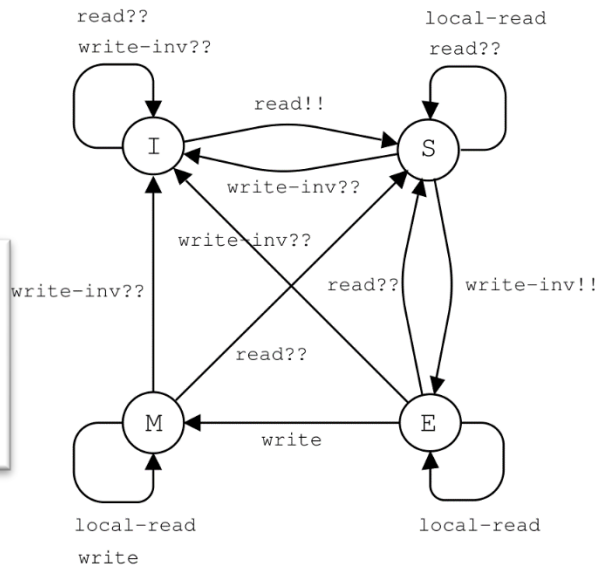
Not more than one process in M or E at the same time:
$U = \uparrow \{(2,0,0,0), (1,1,0,0), (0,2,0,0)\}$ are unsafe states

compute $pred(U)$:

One can show that the finite minimal basis of $pred(R')$ are the minimal elements of

$$\bigcup_{a \in \Sigma} \{c \mid \exists c' \in basis(R') : [(c' = M_a \cdot (c - u_i) + u_j) \vee (c'(j) = 0 \wedge c' + u_j = M_a \cdot (c - u_i) + u_j)] \wedge c(i) \geq 1\}$$

case: $c \xrightarrow{a} c'$ case: $c \xrightarrow{a} (c' + u_j)$



c with $c \xrightarrow{a} (2,0,0,0)$:

$(2,0,0,0)$ for $a \in \{local\text{-}read_M, write_M\}$ (where a_M is the unique action a with sender starting in state M)
$(1,1,0,0)$ for $a = write_E$
impossible for all other actions, since sender does not go to M

c with $c \xrightarrow{a} (2,0,0,0) + u_j$ and $j \neq 0$:

all of these c are already in U, same for $c' = (1,1,0,0)$ and $c' = (0,2,0,0)$

$(2,1,0,0), (2,0,1,0)$ for $a \in \{local\text{-}read_E, local\text{-}read_S\}$
impossible for all other actions: either sender goes to M, or all receivers go to I

Example: Model Checking a Simplified MESI Protocol (II)

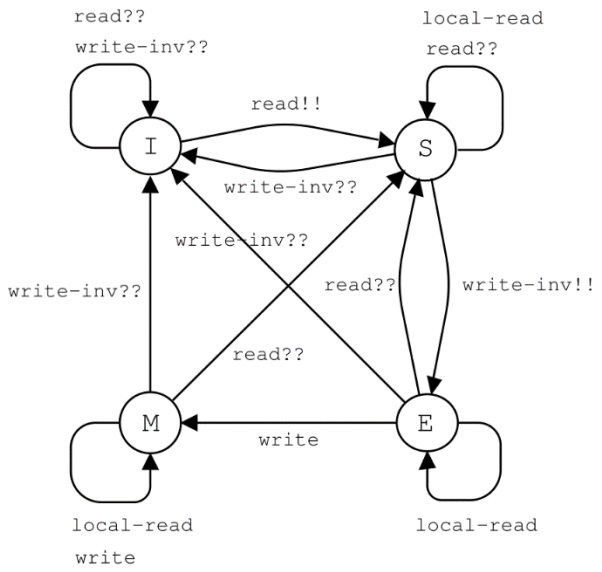
Not more than one process in M or E at the same time:

$U = \uparrow \{(2,0,0,0), (1,1,0,0), (0,2,0,0)\}$ are unsafe states

$pred(U) = U$

$Initial \cap U = \emptyset$

Termination after one step, U is unreachable.



A cache line should not be modified by one process (M) and in shared state for another (S):

$\mathrm{U} = \uparrow \{(1,0,1,0)\}$ are unsafe states

$pred(U) = \uparrow \{(1,0,1,0), (0,1,1,0)\} \supseteq U$

$pred\big(pred(U)\big) = pred(U)$

$Initial \cap pred(U) = \emptyset$

Correctness of the Model Checking Algorithm

We have shown:

- if unsafe states are given as upward-closed set, we can represent them by their finite basis
- we can compute their (repeated) predecessors and represent them by their finite basis
- if we eventually find an overlap with initial states, then unsafe states are reachable
- if we reach a fixed point without such an overlap, then unsafe states are unreachable

To show:

the algorithm always terminates

Proof: With every computation of $pred$, the minimal basis either grows by at least one element or the algorithm terminates. Since the minimal basis is an antichain wrt. our wqo, it can not grow infinitely. Therefore, the algorithm must eventually terminate.

Summary: Decidability Results for Broadcast Protocols

- Counter system is an abstraction that preserves reachability properties and allows us to model check systems with many processes
- A well-quasi-order on states of a parameterized system allows us to represent infinite sets of states by a finite basis
- Together, the counter system and the wqo allow us to decide the PMCP for reachability of arbitrary upward-closed sets

Decidability Results: Reachability Properties in (Pairwise and Asynchronous) Rendezvous Systems

Well-Structured Transition Systems (I)

Well-structured transition systems are a general class of infinite-state transition systems for which the problem of model checking safety/reachability properties is decidable.

They can be used to model certain classes of parameterized systems, like broadcast protocols or rendezvous systems, and give us a decision procedure for checking reachability.

Additionally, they can model other kinds of infinite-state systems, e.g. timed automata or "relational automata" (that have a finite set of variables of type $\mathbb{Q}$ or $\mathbb{Z}$).

Well-Structured Transition Systems (II)

For a (possibly infinite-state) transition system (S, S_0, Σ, Δ), we write $s \to^* s'$ if there exists a path from s to s'.

We say that a wqo $\lesssim \subseteq S \times S$ is compatible with Δ if the following holds:

whenever $s \to s'$ and $s \lesssim t$ then there exists t' with $s' \lesssim t'$ and $t \to^* t'$.

$$\begin{array}{ccc} t & \to^* & t' \\ \rotatebox{90}{$\lesssim$} & & \rotatebox{90}{$\lesssim$} \\ s & \to & s' \end{array}$$

We say $\lesssim$ is strongly compatible with Δ if the above holds with $t \to t'$ instead of $t \to^* t'$.

Examples: Compatibility and Strong Compatibility

Strong compatibility: broadcast protocols

if s, s' and t are states in a broadcast system with $s \preccurlyeq t$ and $s \xrightarrow{a} s'$,
then the state t' we reach with $t \xrightarrow{a} t'$ always satisfies $s' \preccurlyeq t'$:

$$s' = M_a \cdot (s - u_i) + u_j \preccurlyeq M_a \cdot (t - u_i) + u_j = t'$$

Compatibility:

consider a transition system with state set $(\mathbb{N}_0)^2$, i.e., with two counters, with transitions $(n, m) \to (n - 1, m + 1)$ and the wqo

$$(n, m) \lesssim (n', m') \quad \Leftrightarrow \quad n \leq n' \wedge m \leq m' \wedge (n = 0 \Leftrightarrow n' = 0)$$

then $(1,3) \lesssim (3,3)$ and $(1,3) \to (0,4)$. However, $(3,3) \to (2,4)$ with $\neg\big((0,4) \lesssim (2,4)\big)$.

Instead, we have $(3,3) \to (2,4) \to (1,5) \to (0,6)$ with $(0,4) \lesssim (0,6)$

Well-Structured Transition Systems (III)

An infinite-state labeled transition system (S, S_0, Σ, Δ) is a well-structured transition system (WSTS) if there exists a wqo $\lesssim \subseteq S \times S$ that is compatible with Δ.

Let $R \subseteq S$ and let $pred(R)$ be the set of immediate predecessors of R (with respect to Δ).

If R is upward-closed (wrt. $\lesssim$), then for the sequence $R_0 \subseteq R_1 \subseteq \cdots$ with $R_0 = R$ and $R_{i+1} = R_i \cup pred(R_i)$ there exists $k \in \mathbb{N}$ with $R_{k+1} = R_k$. We write $pred^*(R)$ for this R_k.

Note: In this sequence, every R_i is upward-closed. Therefore, $pred^*(R)$ is upward-closed.

We say a WSTS has effective pred-basis if there exists an algorithm that takes as input any state $s \in S$ and returns a finite basis of $\uparrow pred(\uparrow s)$.

Observation: For a WSTS with effective pred-basis, we can compute a finite basis of $pred(R)$ for any upward-closed R that is given as a finite basis.

Well-Structured Transition Systems (IV)

We call a set $I \subseteq S$ downward-closed if whenever $s \in I$ and $s' \lesssim s$, then $s' \in I$.

Example: in the counter system of a parameterized system, the set of initial states is always downward-closed.

Note: downward-closed sets do not necessarily have a finite basis.

Theorem:

For a WSTS with effective pred-basis, we can decide reachability of an upward-closed set R from an initial set of states I if either

- I is upward-closed and we can decide emptiness of $U \cap U'$ for any upward-closed sets U, U'

or

- I is downward-closed and we can decide emptiness of $D \cap U$ for any downward-closed set D and upward-closed set U

Example: Intersection of Upward- and Downward-closed Sets

For broadcast protocols with wqo $\preccurlyeq$, emptiness of $U \cap U'$ for upward-closed sets U, U' is decidable:

$U \cap U'$ is non-empty if there is $c \in basis(U), c' \in basis(U')$ and d with $c \preccurlyeq d$ and $c' \preccurlyeq d$

if $c = (c_1, \dots, c_k)$ and $c' = (c'_1, \dots, c'_k)$, then this holds for $d = (\max(c_1, c'_1), \dots, \max(c_k, c'_k))$

(which also means that two upward-closed sets wrt. $\preccurlyeq$ never have an empty intersection)

A downward-closed set I wrt. $\preccurlyeq$ can be defined by a finite set of maximal elements from $(\mathbb{N}_0 \cup \infty)^k$, similar to a finite basis. E.g., $(\infty, 0, \dots, 0)$, where the first component of the vector corresponds to the local initial state of the processes.

Then the intersection $I \cap U$ with an upward-closed set U is non-empty if there exist such a maximal element c and a $c' \in basis(U)$ and d with $c' \preccurlyeq d \preccurlyeq c$.

Such a d exists if and only if one if the elements of $basis(U)$ is in I.

Counter Systems of other Parameterized Systems

For systems with pairwise rendezvous ($card = \{1\}$) or asynchronous rendezvous ($card = \{0,1\}$), the counter system $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ of $sys(P, card)$ is defined in the same way as for broadcast protocols.

Computing the effect of a transition is even simpler:

- a pairwise rendezvous transition $c \xrightarrow{a} c'$ based on $q_i \xrightarrow{a!} q_j$ and $q_k \xrightarrow{a?} q_l$ can be computed by letting $c' = c - u_i + u_j - u_k + u_l$, with the condition that $c(q_i) \geq 1$ and $c(q_k) \geq 1$
- an asynchronous rendezvous transition $c \xrightarrow{a} c'$ is either computed in the same way, or by letting $c' = c - u_i + u_j$, with the condition that $c(q_i) \geq 1$ and $\nexists\, q_k$ with $q_k \xrightarrow{a?} q_l$ and $c(q_k) \geq 1$

Parameterized Systems as WSTSs

Theorem:
If $P = (Q, Q_0, \Sigma, \delta, \lambda)$ is a process based on actions Σ_{int} and Σ_{sync}, and $card \subseteq \mathbb{N}_0$ is an interval, then the counter system of $sys(P, card)$ with the wqo $\preccurlyeq$ is a WSTS with effective pred-basis.

Corollary:
The counter system of a parameterized system with pairwise rendezvous or asynchronous rendezvous synchronization is a WSTS with effective pred-basis wrt. $\preccurlyeq$.

Parameterized Systems with Multiple Communication Primitives

Theorem:
If $M_1 = (S, S_0, \Sigma_1, \Delta_1)$ and $M_2 = (S, S_0, \Sigma_2, \Delta_2)$ are transition systems with $\Sigma_1 \cap \Sigma_2 = \emptyset$ and both M_1 and M_2 are WSTSs with effective pred-basis wrt. the same wqo $\lesssim$, then the transition system $M_1 \cup M_2 = (S, S_0, \Sigma_1 \cup \Sigma_2, \Delta_1 \cup \Delta_2)$ is a WSTS with effective pred-basis wrt. $\lesssim$.

Corollary:
If $M_i = (S, S_0, \Sigma_i, \Delta_i)$, for $i \in \{1,2,3\}$ are transition systems based on pairwise rendezvous, asynchronous rendezvous and broadcast communication, respectively, then $M_1 \cup M_2 \cup M_3$ is a WSTS with effective pred-basis wrt. $\preccurlyeq$.

Decidability Results: Beyond Reachability Properties

Beyond Reachability Properties

What we have seen until now gives us decidability of the PMCP for

- arbitrary finite-state processes and
- a wide class of communication primitives (arbitrary combinations of broadcast, pairwise rendezvous, asynchronous rendezvous, and other communication primitives),

but the class of properties is restricted to (un)reachability of a set of states.

There are several other important classes of properties that we would like to be able to specify and reason about, e.g. properties of the form

- „if A happens, then B will not happen afterwards“
- „whenever A happens, then B will happen within k steps“
- „whenever A happens, then B will happen at some point in the future“
- „at every point of the execution, A will happen again in the future“

Safety and Liveness Properties (I)

The mentioned properties can be assigned to two classes of properties:

- „if A happens, then B will not happen afterwards“
- „whenever A happens, then B will happen within k steps“

} safety properties

- „whenever A happens, then B will happen at some point in the future“
- „at every point of the execution, A will happen again in the future“

} liveness properties

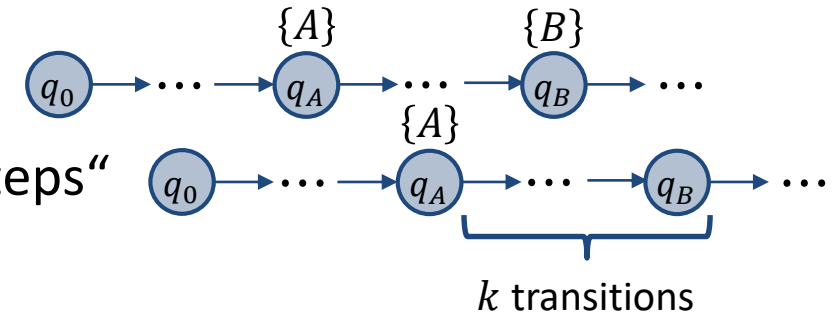
? Why is the second a safety property, but the third a liveness property?

Safety and Liveness Properties (II)

A property of the (parameterized) system is a property that holds for all runs of the system.

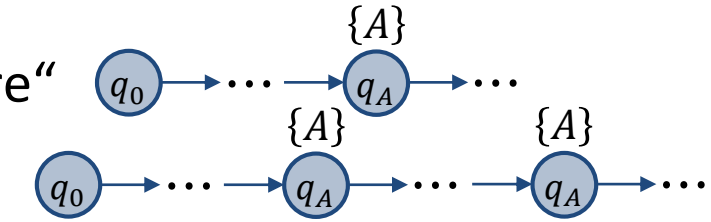
Safety properties are properties that are violated after finite time, even in infinite runs:

- „if A happens, then B will not happen afterwards“
- „whenever A happens, then B will happen within k steps“



Liveness properties can only be violated in infinite time

- „whenever A happens, then B will happen at some point in the future“
- „at every point of the execution, A will happen again in the future“



For finite state transition systems, model checking is decidable for both safety and liveness properties.

For many parameterized systems, the difference between safety and liveness determines whether the PMCP is decidable or not.

Checking Safety Properties: Finite-State Automata (I)

In model checking, properties to be checked are often encoded as automata that act as a memory of events that are important for the specification.

A finite-state automaton is a tuple $(Q, q_0, \Sigma, \delta, F)$ where

- Q is a finite set of states
- $q_0 \in Q$ is the initial state
- Σ is an input alphabet
- $\delta \subseteq Q \times \Sigma \times Q$ is a transition relation
- $F \subseteq Q$ is a set of accepting states

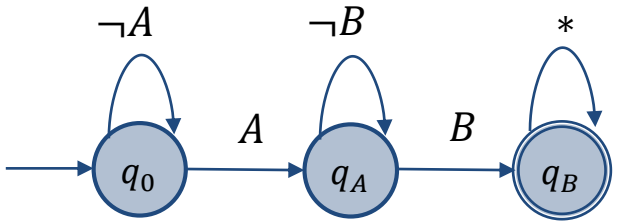
A run of the automaton is a finite sequence $q_1 a_1 q_2 a_2 \dots a_n q_{n+1}$ in $(Q\Sigma)^* Q$ with $q_1 = q_0$ and $(q_i, a_i q_{i+1}) \in \delta$ for $i \in \{1, \dots, n\}$. We say it is a run on the word $w = a_1 a_2 \dots a_n \in \Sigma^*$.

The run is accepting if $q_{n+1} \in F$. A word $w \in \Sigma^*$ is accepted by the automaton if there exists an accepting run of the automaton on w.

Checking Safety Properties: Finite-State Automata (II)

Suppose we want to check the property „if A happens, then B will not happen afterwards“, where A and B are atomic propositions (in state labels) of the system we want to check.

To this end, we first need an automaton that accepts all words that violate the property:



We write propositional formulas on edges to represent the set of subsets of AP that satisfy this formula, e.g.,
$\neg A$ stands for $\{\emptyset, \{B\}\}$,
A stands for $\{\{A\}, \{A, B\}\}$,
$A \wedge B$ stands for $\{\{A, B\}\}$

Formally:

$Q = \{q_0, q_A, q_B\}$

$\Sigma = 2^{AP}$ for $AP = \{A, B\}$

$$\delta = \begin{cases} \{(q_0, a, q_0) \mid A \notin a\} \cup \{(q_0, a, q_A) \mid A \in a\} \\ \cup \{(q_A, a, q_A) \mid B \notin a\} \cup \{(q_A, a, q_B) \mid B \in a\} \\ \cup \{(q_B, a, q_B) \mid a \in \Sigma\} \end{cases}$$

$F = \{q_B\}$

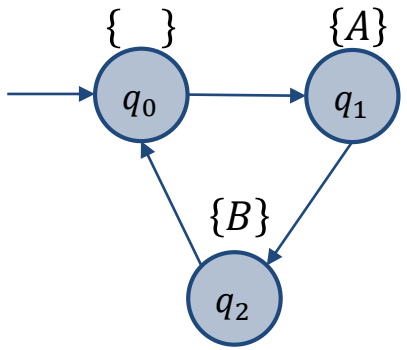
Checking Safety Properties: Finite-State Automata (II)

To check whether a safety property is satisfied by an LTS M, we construct the composition of M with the automaton that accepts all words that violate the property.

The composition $M \times A$ of an LTS $M = (Q, Q_0, \Sigma, \delta, \lambda)$ with
a finite-state automaton $A = (Q_A, q_{A,0}, \Sigma_A, \delta_A, F_A)$, where $\lambda: Q \to 2^{AP}$ and $\Sigma_A = 2^{AP}$,
is the transition system $(Q \times Q_A, Q_0 \times \{q_{A,0}\}, \Sigma, \Delta)$ with
$((q, q_A), a, (q', q'_A)) \in \Delta$ if $(q, a, q') \in \delta$ and $(q_A, \lambda(q), q'_A) \in \delta_A$

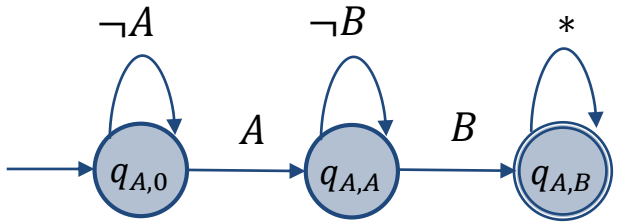
Example:

M

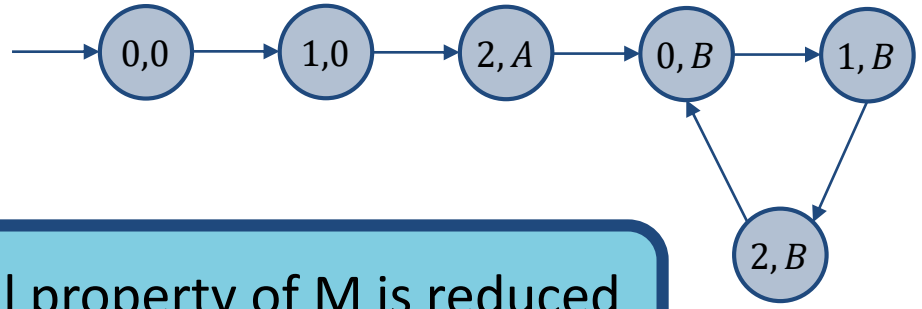


(omitting actions)

A



$M \times A$



Checking the original property of M is reduced to checking state reachability in $M \times A$

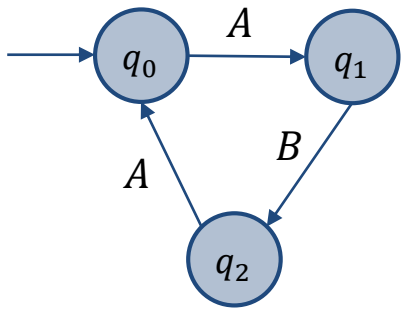
Checking Safety Properties: Finite-State Automata (III)

Instead of the state labels, the automaton can also read the actions of an LTS. This allows us to use the automaton to check properties of the actions along runs of the system.

The composition $M \times A$ of an LTS $M = (Q, Q_0, \Sigma, \delta, \lambda)$ with a finite-state automaton $A = (Q_A, q_{A,0}, \Sigma_A, \delta_A)$, where $\Sigma_A = \Sigma$, is the transition system $(Q \times Q_A, Q_0 \times \{q_{A,0}\}, \Sigma, \Delta)$ with $((q, q_A), a, (q', q'_A)) \in \Delta$ if $(q, a, q') \in \delta$ and $(q_A, a, q'_A) \in \delta_A$

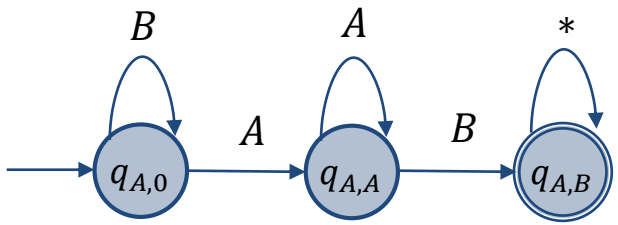
Example:

M

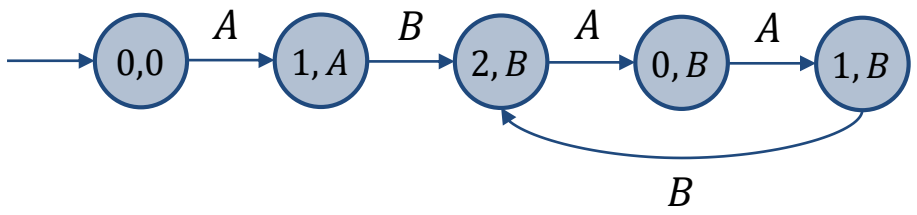


(omitting state labels)

A



$M \times A$



Checking Safety Properties: Finite-State Automata (IV)

Regardless of whether the automaton reads state labels or actions, it reduces the checking of arbitrary safety properties to checking reachability of states in the product $M \times A$:

if in $M \times A$ we can reach a state (q, q_A) where $q_A \in F$, then the path to that state corresponds to a run of the system M that violates the safety property

Checking Arbitrary Safety Properties in Parameterized Systems

For an automaton that reads actions of the system, we can construct the composition $M \times A$ also if M is the counter system of a parameterized system.

For an automaton that reads the state labels, there is a problem: we have omitted the state labels in the counter system, because

- if we just keep the labels of all processes, then the label of the parameterized system depends on the size of the system (and we would need a "parameterized automaton" to read it)
- if we modify the labels, then this modification will influence what kind of properties we can still reason about

This is usually solved by keeping only the state labels of a fixed number of processes (usually one or two) in the composition. We will see later how to do that.

Checking Arbitrary Safety Properties in Parameterized Systems

Even for the case of specifications on actions, where $M \times A$ can be constructed directly, it is not immediately clear if the PMCP is still decidable, since we don't know that the resulting composed system is still a WSTS.

Theorem:

Let $M = (S, S_0, \Sigma, \Delta)$ be a WSTS with effective pred-basis wrt. $\lesssim$, and $A = (Q, q_0, \Sigma, \delta)$ a finite-state automaton. Then $M \times A$ is a WSTS with effective pred-basis wrt. the wqo $\lesssim_A$ defined by

$$(s, q) \lesssim_A (s', q') \quad \Leftrightarrow \quad s \lesssim s' \wedge q = q'$$

Review/Lessons Learned in this Lecture

Topics:

- Model Checking Broadcast Protocols
- Example application of model checking algorithm
- Parameterized model checking:
  - for rendezvous systems
  - for general safety properties

Techniques:

- Well-structured Transition Systems (WSTSs)
- Expressing safety properties as finite automata
- Parameterized model checking based on finite automata